

Spring 2026 MSML606 HW3 (deadline: 3/14, Saturday)

Overview

This assignment focuses on implementing a hash table from scratch. You will analyze the performance of your hash table and compare search time across different distributions.

GitHub Link: [GitHub Classroom Assignment](https://classroom.github.com/a/c8J6iDhU) (https://classroom.github.com/a/c8J6iDhU)

Instructions:

- Do not directly copy content from textbooks, other individuals' work, AI-generated outputs, online sources, or any material that isn't your own. While collaboration and consulting various resources are encouraged, it's essential that you express your understanding in your own words, reflecting your personal learning and insights.
- Add justifications/explanations to all answers, as there will be points awarded for that. Please write in your own words.

Important!

Before submitting your work, review the external-source policy in the syllabus. **You must either (1) cite any external sources you used, including AI tools, or (2) explicitly state that you did not use any. Submissions without this AI usage statement will receive no points!!!**

Part I: Hash table implementation

Deliverables: Submit code and report via GitHub. As always, we will check the commit history and will consider the entire assignment pushed all at once a red flag. Please write your comments in the source code in your own words.

Your submission should include:

- **Homework3.py** - Source code with both your hash table implementation
- **report.pdf** or **report.docx** - Analysis report

Problem 1. Implement a hash table. Tasks:

- Store key-value pairs in an array-based implementation. (e.g., Python's list, Array module (import array), NumPy arrays, Pandas). Do not use Python's built-in dictionary or any other libraries/modules that support built-in hashing!
- Explicitly compute the hash value using a self-design hash function:
 - Division method

- Multiplication method
- Collision resolution method (choose one):
 - Chaining
 - Open addressing (linear probing, quadratic probing, or double hashing)

Required operations for your Hash Table data structure:

- search(key): retrieve a value associated with a key
- insert(key, value): insert key-value pairs into the hash table
- delete(key): remove a key-value pair from the hash table
- dynamic resizing (optional for open addressing):
 - If you implement open addressing, you may choose to include dynamic resizing to maintain efficiency
 - If you opt not to implement resizing, discuss its possible limitations in your report

Problem 2. Performance Analysis

- Load factor vs. lookup time (fix 2-3 table sizes)
 - Plot load factor vs. average search time across different table sizes
 - Use both successful and unsuccessful searches to evaluate worst-case performance
 - Use timer (e.g., `time.perf_counter()`) to measure average search time for a randomly selected key
- Key distribution comparisons (use fixed table size and load factor):
 - Compare search time across different distributions
 - Uniform distribution
 - Skewed distribution
 - Sequential keys
 - Memory overhead analysis:
 - If you are using open addressing, measure average probe length for both search and insertion
 - If you are using chaining, measure average chain length per slot and maximum chain length

Problem 3. Report

Requirements: You must write a report that includes

- Implementation Details: brief explanation of how your code works
- Performance Comparison
 - Search time across different distributions
 - Memory overhead analysis
 - Measurements of chain/probe lengths
 - Use plots/tables to summarize results of Problem 2
- Observations & Discussion

- Under what load factors does each method degrade? What are the memory trade-offs? Discuss both chaining and open addressing (regardless of which you chose to implement)
- Discuss how different distributions may influence the performance

Rubric for Part I: (Maximum Score: 30 Marks)

Problem	Criteria	Marks
Problem 1: Hash Table Implementation & Collision Resolution	- Correctly implements <i>search(key)</i>, <i>insert(key, value)</i>, and <i>delete(key)</i> without using built-in dictionaries (2.5 marks) - A self-designed hash function is provided that works for the expected key types (2.5 marks) - Both division and multiplication methods are implemented correctly (2.5 marks) - Clearly implements a collision resolution method (chaining or open addressing) and discusses (or implements) any dynamic resizing if applicable (2.5 marks)	10
Problem 2: Performance Analysis of the Hash Table	- Conducts experiments on 2-3 fixed table sizes to measure average search time using both successful and unsuccessful searches (2 marks) - Uses timers (e.g., <code>time.perf_counter()</code>) correctly, and results are presented clearly via graphs or tables (2 marks) - Measure average search time for a randomly selected key (2 marks) - Compares search times using different distributions (uniform, skewed, sequential) (2 marks) - For chaining: measures average chain length and maximum chain length. For open addressing: measures average probe lengths for search and insertion (2 marks)	10
Problem 3: Report	- Brief descriptions of the hash table implementation including hash function and collision resolution and any design decisions (2.5 marks) - Explain usage differences and any trade-offs between the open addressing and chaining approaches (2.5 marks) - Include graphs/tables (e.g., load factor vs. lookup time, memory overhead) and discusses trends or anomalies (2.5 marks) - Clearly documents insights regarding key distributions, load factors, and any issues encountered (e.g., dynamic resizing, RecursionError) (2.5 marks)	10